PROJECT REPORT OF CO-OP PROJECT AT INDUSTRY (MODULE-11)

On

# TravelX

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING


Submitted by:
Ujjwal Mutneja
Vaibhav Tiwari
Sahibjot Singh Benipal
Harshita Bansal

Supervised By:
Dr Ashish Kumar
Dr. Gifty Gupta

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY

**CHANDIGARH-PATIALA NATIONAL HIGHWAY, RAJPURA, PUNJAB, INDIA**

CHITKARA UNIVERSITY

# CONTENTS


| S.No. | Title | Page No. |
|---|---|---|
| | Declaration | 1 |
| | Acknowledgement | 2 |
| | List of Figures and Tables | |
| | Abstract | 3 |
| 1 | Introduction | 4-5 |
| 2 | System Requirements | 6 |
| 3 | Methodology | 7-9 |
| 4 | Software Design (architecture/UML/DFD/UI-UX whichever is relevant to your project ) | 10-16 |
| 5 | Implementation Details (Frameworks, Database design and schema) | 17 |
| 6 | Major Findings/Outcomes/Output/Results | 18-27 |
| 7 | Limitations | 28 |
| 8 | Conclusion and Future Scope | 29-30 |
| | References | |
| | Appendices | |
| | | : |

# DECLARATION

We hereby declare that the project work titled “TravelX”, submitted as part of Bachelor’s degree in CSE, at Chitkara University, Punjab, is an authentic record of our own work carried out under the supervision of Dr.Ashish Kumar and Dr. Gifty Gupta.

Signature(s)

# ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behavior, and acts during the course of study.

We express our sincere gratitude to all for providing me an opportunity to undergo an integrated project as part of the curriculum.

We are thankful to Dr. Ashish Kumar and Dr. Gifty Gupta for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence, and blessings.

We also extend our sincere appreciation to Dr. Ashish Kumar and Dr. Gifty Gupta, who provided his valuable suggestions and precious time in accomplishing our integrated project report.

Lastly, we would like to thank the almighty and our parents for their moral support and friends with whom we shared our day-to-day experience and received lots of suggestions that improved our quality of work.

Ujjwal Mutneja-2210992478
Vaibhav Tiwari-2210992491
Sahibjot Singh-2210992207
Harshita Bansal-2210990392

# Abstract/Keywords

This project report details the development of a TravelX application using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system enables seamless ride-hailing services by allowing users to book rides, drivers to accept requests, and an admin to oversee operations. The platform includes features such as real-time tracking, ride history, fare estimation, and secure payment integration. Users can log in, manage their profiles, and track ongoing rides, while drivers can update their availability and ride status. The system leverages Tom Tom API for route optimization and Socket.io for real-time communication. By digitizing the ride-booking process, this application enhances user convenience and ensures efficient transportation management.

## Keywords

Ride-Hailing, TravelX , MERN Stack, React, Node.js, Express, MongoDB, Real-time Tracking, Tom Tom API, Socket.io, Payment Integration, ,Driver Dashboard and User Management.

# 1. INTRODUCTION

## 1.1 Background

Due to advancements in technology, many cities across the globe are seeing significant changes in their urban transportation systems. Ride-hailing is now the primary way people commute, due to the numerous advantages they have over traditional taxi services, including convenience, low prices, and fast service.

Currently there are four major problems facing ride-hailing providers: Problems with real-time locations, ride allocation, secure online payments, and driver availability. Fixing these issues is critical to the success of an orderly and timely process when booking an ordered ride.

The application being introduced by this project is called TravelX, and has been developed using the MERN Technology stack with all of the necessary features required to function as a complete ride-hailing application. It has built-in systems for authentication and authorization for riders (the individuals riding), drivers (the individuals who drive), and administrators (the individuals running the system), all of which are role-based.

Users can request and book a ride at the time they wish, and will be able to track their ride while using their debit and/or credit card through the application. Drivers (captains) can accept ride requests, update their status on whether or not they are available to drive, and have tools to make managing their rides easier. Captains will also be able to manage their entire financial relationship with TravelX, which means they will be able to keep a record of all transactions taking place between them and TravelX, manage their wallets, process settlements, and get cash out for all of their completed transactions. Captains, riders (the individuals riding), and TravelX can rate and provide feedback to ensure the quality of service being provided.

In addition to all of the above mentioned features, TravelX will also leverage the TomTom API for optimal route finders, and will use Socket.io to provide real-time data to users of the application.

## 1.2 Problem Statement

Taxi services and other transportation marketplaces have long experienced problems such as, but not limited too:

1. finding a taxi and/or driver in a timely manner
2. no means of being able to track the taxi or how far away the taxi was from a given location (thus providing for uncertainty to the passenger)
3. the uncertainty of what the price of the ride is going to be at the end of the ride (fair estimate)
4. no clear means of communication between the passenger and the driver (placing the safety of both passenger and driver at risk).

The main objectives of the project is to develop a web based ride hailing system that provides users the ability to both book and track their ride(s) in real-time; provide secure digital payment methods; and provide estimated fares prior to booking the ride(s). Through these methods, the application provides an unparalleled level of communication/transparency when it comes to providing an overall positive experience for the user in the ride sharing process.

# 2. System Requirements

### 1. Software Requirements

- **OS (Development):** Windows, macOS, Linux (Ubuntu preferred)
- **OS (Users):** Any modern browser-supported OS
- **Frontend:** React.js (Vite) + TailwindCSS + Icons (FontAwesome/Remixicon)
- **Backend:** Node.js + Express.js
- **Database:** MongoDB with Mongoose
- **Real-time:** Socket.io (live tracking & ride updates)
- **APIs & Services:**
    - TomTom Maps (routing & geolocation)
    - Stripe (payments)
    - Cloudinary (image/document storage)
    - Google OAuth + JWT (authentication)
- **Supported Browsers:** Chrome, Firefox, Safari, Edge

---

### 2. Hardware Requirements

**For End Users (Riders & Captains):**

- Devices: Smartphone / Tablet / PC
- Processor: 1.8 GHz quad-core
- RAM: 2 GB (4 GB recommended)
- GPS: Required for captains
- Network: Stable 4G/5G or Wi-Fi

**For Development / Server:**

- Processor: Intel i5 / Ryzen 5 (4 cores)
- RAM: 8 GB (16 GB recommended)
- Storage: 256 GB SSD
- Architecture: 64-bit syst

# 3.Methodology

The development of the Riding Application follows a structured approach leveraging modern web technologies and real-time communication protocols. The methodology dictates the use of a modular, component-based frontend and a robust, scalable RESTful backend to ensure a seamless experience for both Riders and Captains (drivers).

## 3.1. System Architecture (Client-Server)

The Riding Application uses a Client-Server Architecture (MERN - MongoDB, Express.js, React.js and Node.js) with an integrated real-time event-driven layer.

* Client-Side (Front End) - The Front End is the user interface that collects input from the user (e.g., pick up and drop off locations), displays in real-time an interactive map, and provides the user with the ability to track their ride.

* Server-Side (Back End) - The Back End is where all of the business logic is processed, securely authenticated and interacts with 3rd party APIs to retrieve and write data to and from the database as well as process the rides.

* Real-Time Communication Layer - This layer enables bi-directional web sockets to synchronize data in real-time between the back end and both the rider and captain (driver).

## 3.2. Technology Stack

The technology selected for the project provides high performance, secure, and maintainable applications.

- Front End - The front end is built using React.js and Vite allows for fast build times and hot module replacement.
- Styling & UI: Tailwind CSS is utilized for a utility-first, highly responsive design, while GSAP (GreenSock) is incorporated to provide smooth, premium micro-animations.
- Backend Framework: Built on Node.js and Express.js to handle API requests efficiently using non-blocking I/O.
- Database: MongoDB (accessed via Mongoose ODM) is used as a NoSQL database, allowing flexible, document-based storage for Users, Captains, and Ride histories.
- Real Time Engine: Socket.io allows for low latency, event based communication, very important for real time operations.
- Mapping and Routing: Integrated with TomTom Maps API and React - Leaflet for accurate geo-coordinates, routing distance, estimated time of arrival and dynamically rendered map display.

## 3.3. Development Approach and Workflows

The project was developed using a logical, iterative process.

Component Based Design - The UI has been broken down into modular, reusable React components, allowing for the code to be clean and easy to maintain.

RESTful API Principles - The backend routes were developed following regular operations models, (e.g. /users/ login, '/rides/get-fare), leveraging a standard HTTP method and JSON payload.

Security and Authentication Workflow - Stateless Authentication is achieved via use of JSON Web Tokens (JWTs). Passwords are hashed cryptographically with Bcrypt prior to insertion in the database. Google OAuth 2.0 (via Passport.js) provides users with a seamless SSO experience.

## 3.4. Key Functional Implementations

**Fare Calculation & Routing** - When the user inputs their pick-up & destination the frontend requests the coordinates. The backend in Node.js then calculates the best route by passing the information gathered from the mapping API to assess how much distance and time it would take for the trip. Using base rates, time, distance and vehicle type, the backend then generates a dynamic fare for the ride.

**Calculating fares and routing drivers:** After entering the pickup point and destination in the client-side app, coordinates for those locations are requested from the server with Node.js. The coordinates are processed using the mapping API and optimal distance and travel time are returned so that fare can be generated dynamically based on the base fare, travel time, distance, and type of vehicle.

**Matching drivers with riders**: Once a rider has requested a ride, the backend will use Socket.io to notify all available drivers ("Captains") in the vicinity of the ride being requested by sending out a `ride_requested` event within a specific geographical radius from the rider's pickup location.

**Tracking driver's live location**: Once a ride has been accepted by a driver, that driver's mobile device will continue to send their current GPS location to the server at regular intervals. The server will subsequently send those updated GPS coordinates to the frontend interface of the rider in real-time, allowing the rider to see their location on the Leaflet map.

### 💰 Wallet System & Fare Settlement

The system maintains a **digital wallet** for both riders and drivers:

- Riders can pay via wallet, card, or cash.
- After ride completion, fare is automatically deducted.
- Driver earnings are credited to their wallet after deducting platform commission.
- Admin can trigger payouts or settlements to driver bank accounts.

This ensures transparency and seamless financial flow within the platform.

## 📊 Driver Analytics & Performance Monitoring

The platform tracks detailed analytics for each driver, such as:

- Total number of rides
- Income generated
- Ride acceptance rate and cancellations
- Average rating by riders
- Active hours and peak performance

These insights help in:

- Incentive computation
- Performance assessment
- Service improvement

## 🛠️ Admin Panel & Captain Management

With this admin panel, you can manage and monitor your system:

The Customer (Driver) Approval System - When a new driver signs up, the system will send the new driver's profile (including documents, vehicle details, and proof of identity) to the administrator for verification and then the administrator will review and approve or deny the application. When an administrator approves the application, the driver will be able to go online.

Complete Captain View - The administrator can access all of the information about a driver through the driver profile, which includes:

- Personal & Vehicle Information
- Document Verification Status
- Ride History
- Earnings & Wallet Balance

Ride Monitoring & Administration - The administrator can monitor rides that are in progress, resolve disputes, and manage all back-end operations.

# 4. Software Design

The software design of the Riding App is structured to ensure real-time responsiveness, modularity, and high security. It encompasses system architecture, data flow representations, UML sequences, and a strong focus on intuitive User Interface/User Experience (UI/UX) design.

### 4.1. System Architecture

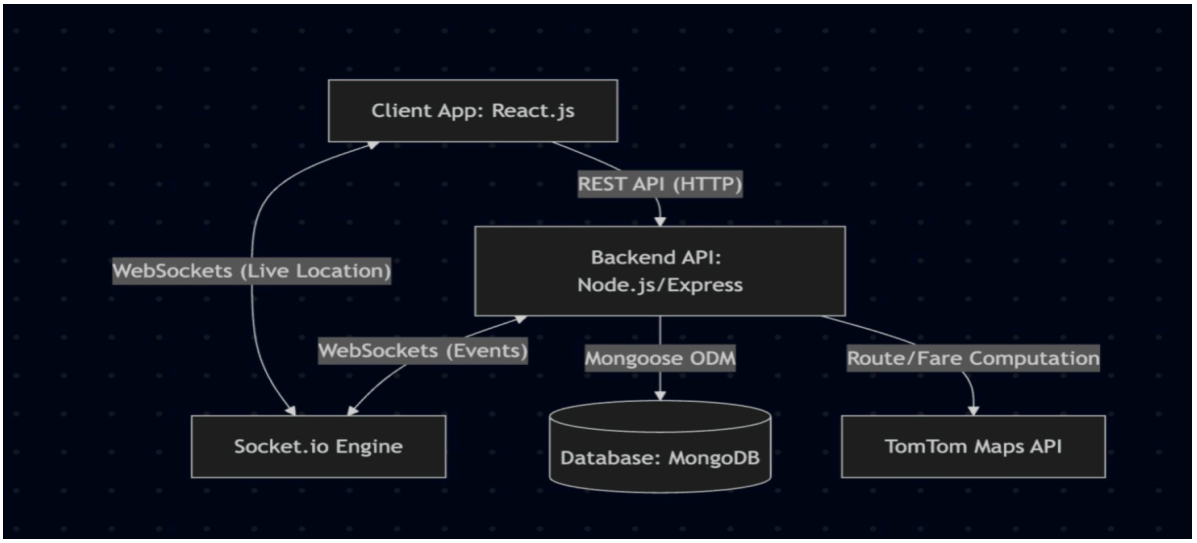
The App has been built and employs a highly decoupled 3 tier Architecture, including a real time communications node and third party APIs, as follows:

- Presentation Layer (Frontend) – React.js has been used to develop the Presentation Layer, which interfaces primarily with mapping, form validations, and intent event listeners, providing the end user with immediate visual feedback when using the App.

- Application Layer (Backend) – The Node.js + Express.js server acts as the primary controller for the Backend Application Layer and will provide the following functions: user authentication (a combination of both JWT and OAuth), interface with the TomTom Maps API to implement routing algorithms, and manage HTTP state.

- Data Layer – The Data Layer utilizes MongoDB to store user profiles, information about each Captain, coordinates and historical ride data.

- WebSocket Layer – A separate instance of a socket.io engine runs in parallel with the HTTP server to maintain persistent connections with users for live GPS tracking of their ride and real-time ride broadcasting.



## 4.2. UML Diagrams

### A. Use Case Diagram

The system caters to two primary actors: the Rider and the Captain (Driver), alongside backend automated processes.

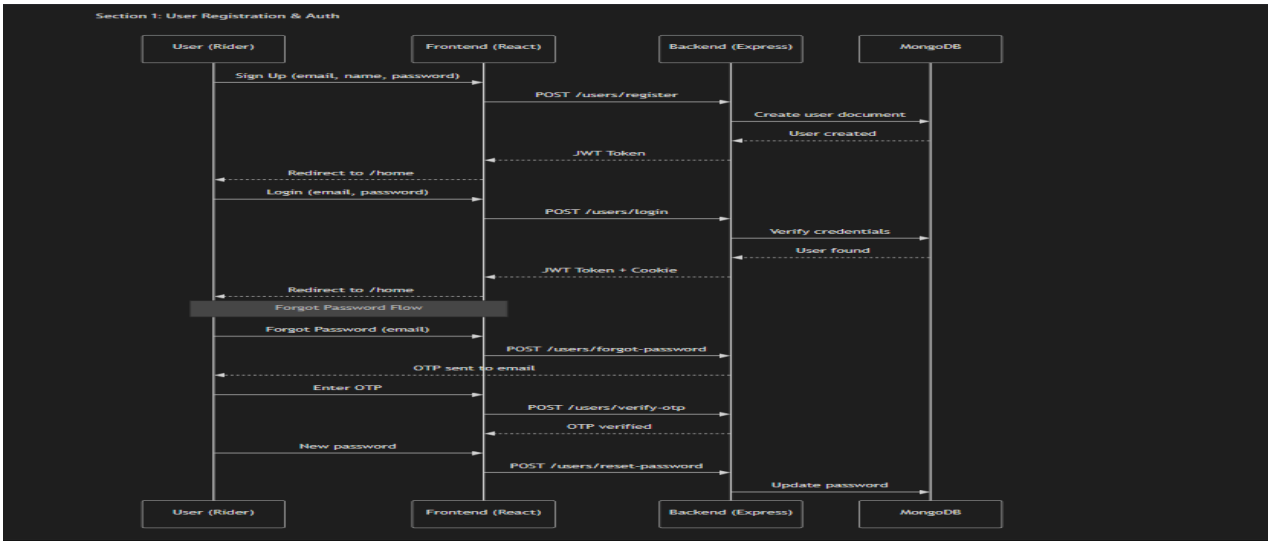
Rider Use Cases: Register/Login via Google OAuth, Input Pickup/Drop-off, View Estimated Fare, Request Ride, Cancel Ride, Track Captain Live.

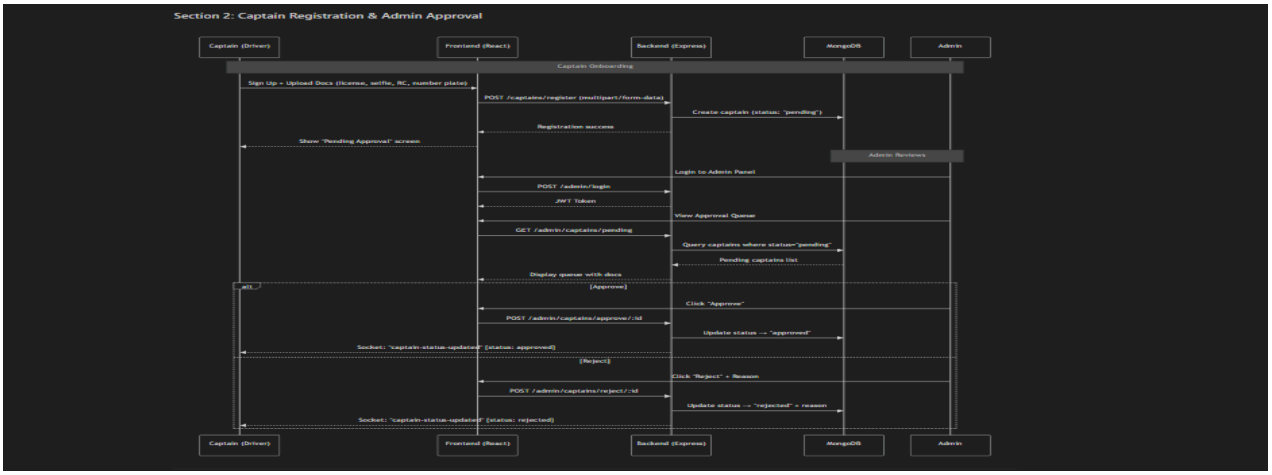
Captain Use Cases: Register/Login, Go Online/Offline, Receive Ride Broadcasts, Accept/Reject Rides, Navigate to Rider, Complete Ride.

System Use Cases: Manage Sessions, Calculate Dynamic Fares, Broadcast Socket Events.

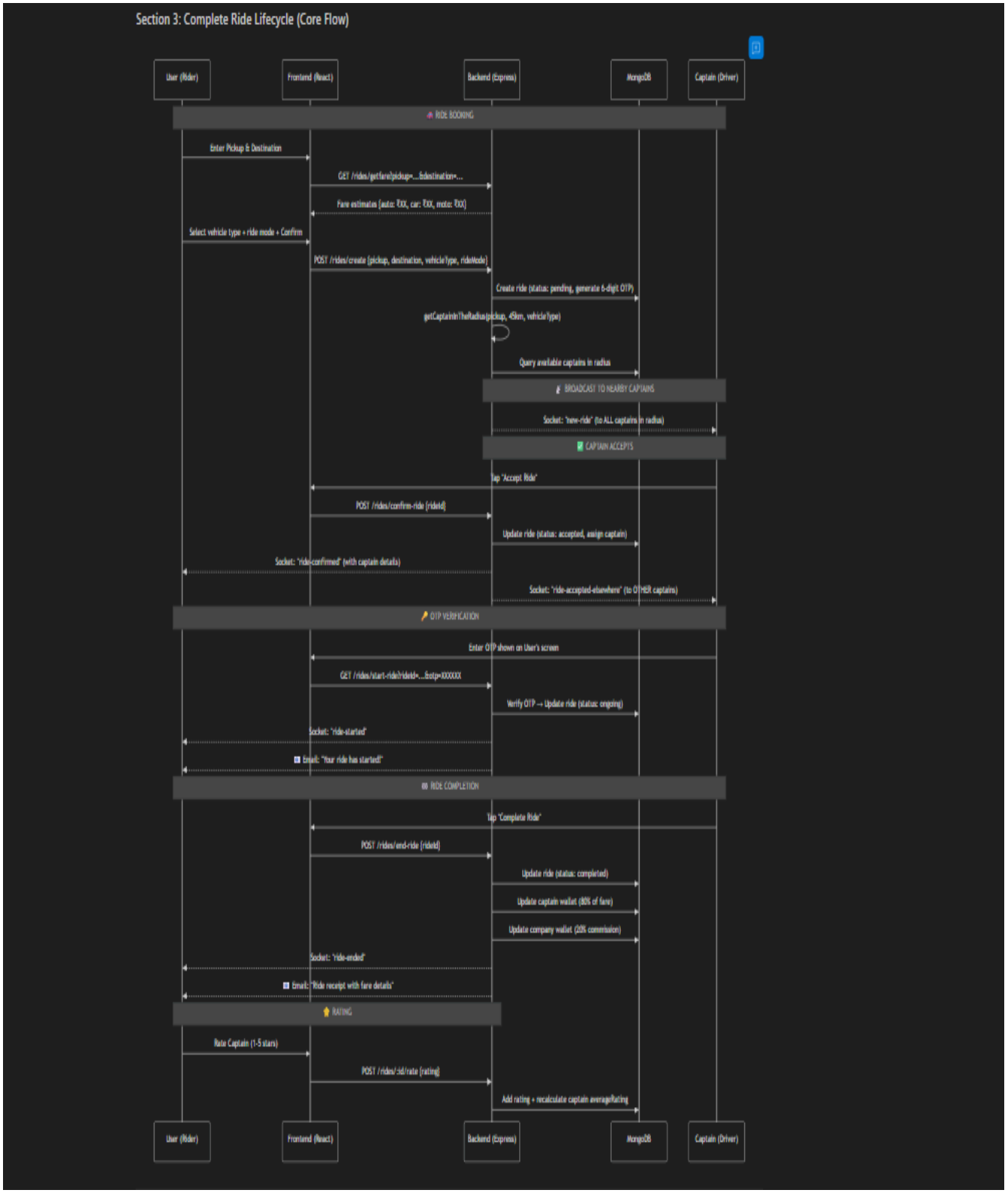
### B. Sequence Diagram (Ride Booking Flow)

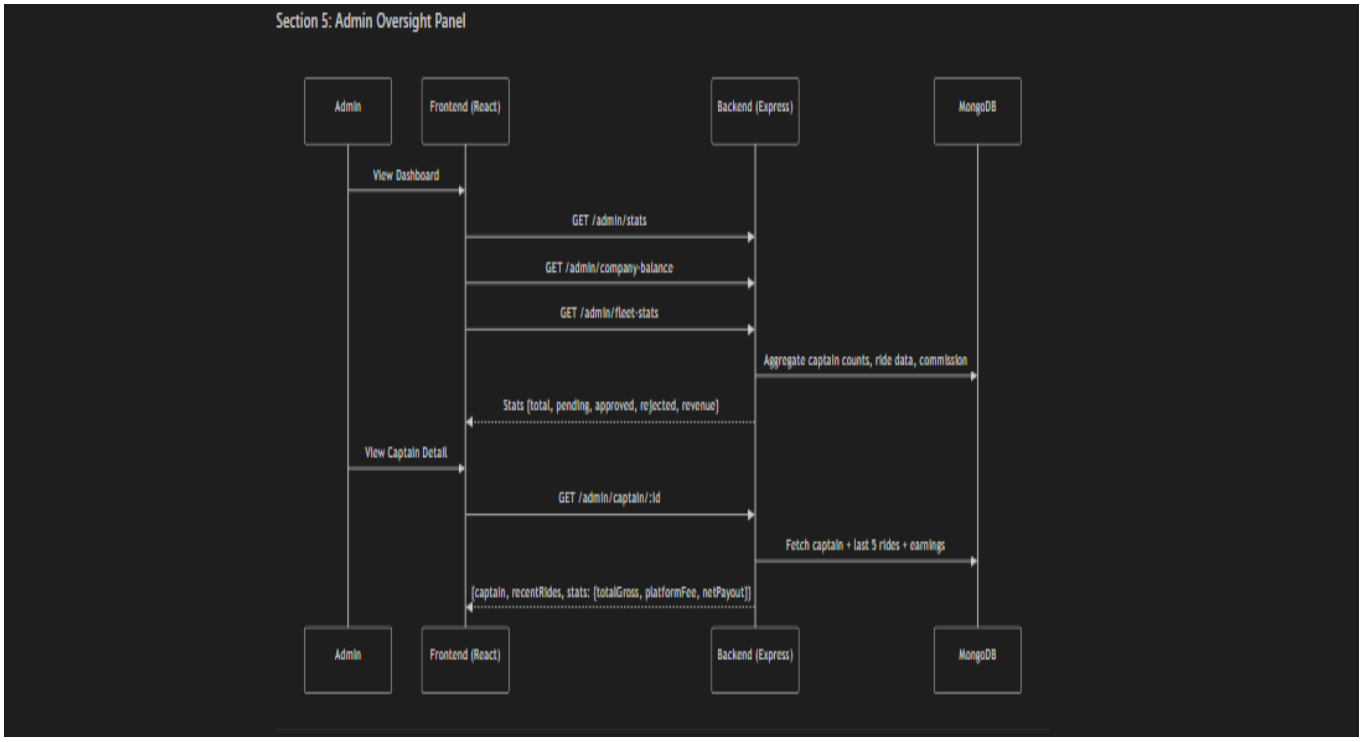
The sequence diagram illustrates the chronological flow of messages between system components during a core ride-booking scenario.

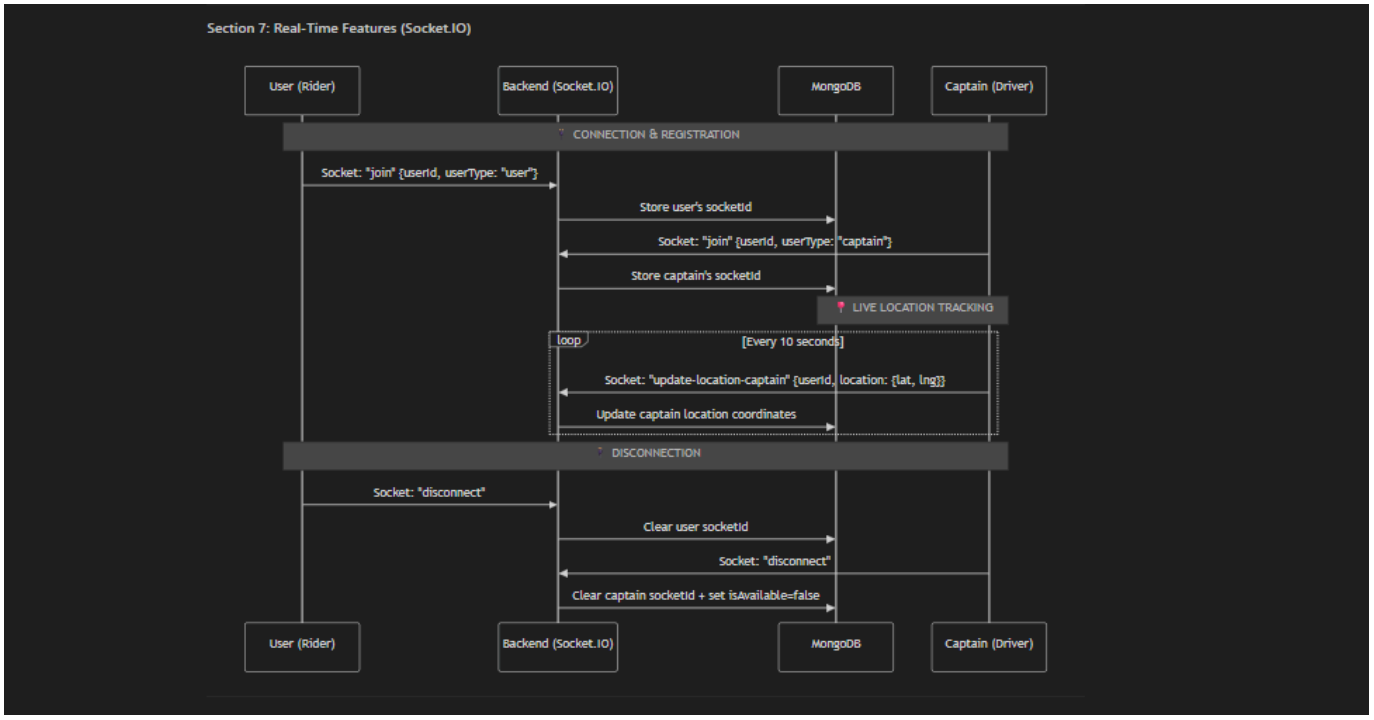


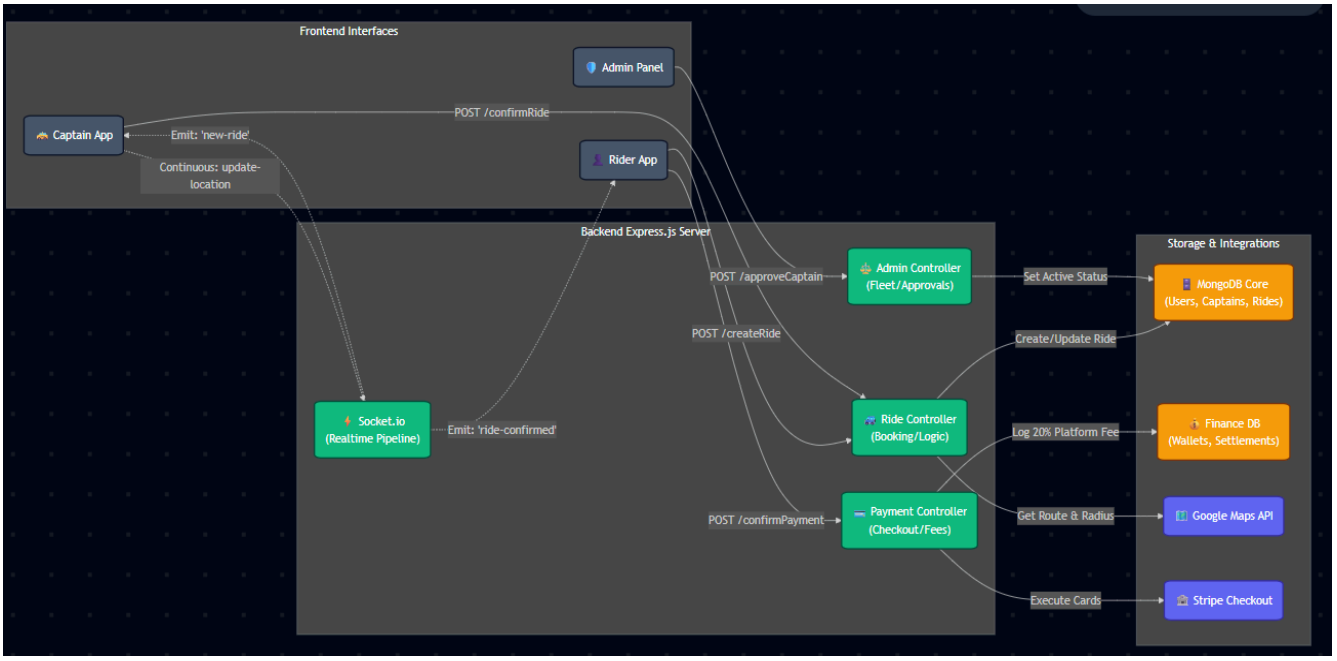
CHITKARA UNIVERSITY




Section 5: Admin Oversight Panel
Admin
Frontend (React)
Backend (Express)
MongoDB
View Dashboard
GET /admin/stats
GET /admin/company-balance
GET /admin/fleet-stats
Aggregate captain counts, ride data, commission
Stats {total, pending, approved, rejected, revenue}
View Captain Detail
GET /admin/captain/:id
Fetch captain + last 5 rides + earnings
{captain, recentRides, stats: {totalGross, platformFee, netPayout}}
Admin
Frontend (React)
Backend (Express)
MongoDB


Section 7: Real-Time Features (Socket.IO)
User (Rider)
Backend (Socket.IO)
MongoDB
Captain (Driver)
CONNECTION & REGISTRATION
Socket: "join" {userId, userType: "user"}
Store user's socketId
Socket: "join" {userId, userType: "captain"}
Store captain's socketId
LIVE LOCATION TRACKING
loop
[Every 10 seconds]
Socket: "update-location-captain" {userId, location: {lat, lng}}
Update captain location coordinates
DISCONNECTION
Socket: "disconnect"
Clear user socketId
Socket: "disconnect"
Clear captain socketId + set isAvailable=false
User (Rider)
Backend (Socket.IO)
MongoDB
Captain (Driver)

### 4.3. Data Flow Diagrams (DFD)




#### 4.3.1. External Entities

These are the external elements that interact with the Travelx system, representing where the data comes from and where it ultimately goes.

**Rider (User)**: The individual requesting a ride, inputting destinations, and initiating payments.

**Captain (Driver)**: The driver accepting ride requests, sharing live GPS coordinates, and receiving payouts.

**System Admin**: The management figure who handles fleet verification, monitors system analytics, and oversees operations.

**External Payment Gateway (Stripe):** The 3rd party API responsible for securely processing credit card/online payments.

**Map/Routing Service (TomTom API):** Translates physical addresses to Geo-coordinates (Geocoding) and assists in calculating live paths and ETA.

#### 4.3.2. Data Stores (Databases/Collections)

The persistent storage layers inside your MongoDB database where data comes to rest.

**D1: Users DB:** Stores rider profiles, authentication credentials, and active WebSocket connection IDs (socketId).

**D2: Captains DB:** Stores driver profiles, vehicle information, verified documents, active statuses, virtual wallet balances, and current geographical coordinates (lat/lng).

**D3: Rides DB:** The core ledger storing individual ride lifecycles (pending, accepted, completed), geospatial mapping points (pickup/destination), calculated fares, and payment constraints.

**D4: Settlement Ledger:** Stores records of cashout requests, historical earnings, and masked banking details.

**D5: Company Central** Wallet: A single operational account tracking total commissions earned and overall platform revenue.

#### 4.3.3. System Processes (The core actions)

These represent the backend logic and the transformation of the data within your Node.js/Express system.

**Process 1: Authentication & Registration Management**

Creates user and captain accounts.

Authenticates individuals mapping their active session to Socket.io endpoints.

Handles multipart form data for driver verification documents.

**Process 2: Ride Matching & Geospatial Dispatching**

Calculates dynamic fares based on route limits.

Runs the $geoWithin algorithm to find "approved" and "available" Captains within a 45km radius.

Broadcasts ride requests instantly to matched drivers using WebSockets.

**Process 3: Real-Time Telemetry & Tracking**

Continuously polls driver device GPS.

Translates map vectors and dynamically updates the TomTom Live Canvas on the Rider's dashboard.

**Process 4: Payment & Commission Settlement**

Verifies secure payment intents with the Stripe network.

Automatically runs commission logic (e.g., extracting a 20% platform fee).

Increments Captain wallets (for online payments) or debts them (for cash payments) while updating the Company Central Wallet.

**Process 5: Analytics & Overview (Admin/Driver Dashboard)**

Transforms raw database entries into aggregated reports (Lifetime earnings, complete ride counts, charts configuration for the frontend UI).

### 4.3.4. UI/UX Design Strategy

The User Interface and User Experience were heavily prioritized to create a "premium" feel, essential for modern aggregator applications:

**Mobile :** By using Tailwind CSS, we have developed the application with a mobile-first layout in mind, allowing for a more ergonomic touch target (e.g. booking buttons and dragging of the map) for the user's smartphone.

**Interactive Micro Animations:** Using Leaflet, the map spans the majority of the screen mimicking native apps. The UI sits above the map as floating, glass-morphic panels to maximize geographical context.

**Map-Based Interface**: The majority of the interface is a map using Leaflet and mimics the design of native map applications. The UI (user interface) is then placed on top of the map and made up of floating glass-morphic panels that provide greater geographical context.

**Real-Time Feedback Integration:** Throughout the application, we are using React Toastify and custom loading spinners to provide users with real-time feedback regarding the state of the system when waiting for API response or accepting a ride.

# 5. Implementation Details

## 5.1 Database design

MongoDB will create the back end of the ride-hailing system. MongoDB is a NoSQL database that is well known for both scalability and flexibility. The database will have three main collections or core entities that make up the ride-hailing system Admin, Rider, and Driver collections.

With the Admin Collection, the admin's privileges and login information will be stored and enable an admin to manage rides, users, and drivers.

**In the Rider Collection**, the user (rider) account information will include the Rider Identification Number (Rider ID), contact information, ride history, and login information for the rider.

**In the Driver Collection**, the driver information will include the driver's name, vehicle information, availability status, and ride acceptance history.

Rides will be stored in the **rides collection**, and will include ride details (rider ID, driver ID, ride status, fare, timestamp of pickup/drop off location(s)).

Fleet Management & Vetting: **The admin collection** interacts directly with the Driver collection to review and manually change a driver's status (e.g., changing from pending to approved after verifying Cloudinary-hosted documents like licenses and selfies).

Tom Tom API, will provide a graphical representation of ride tracking information and trip history for completed and ongoing rides using a map view.

To manage the platform's complex financial ledger, two additional schemas are enforced:

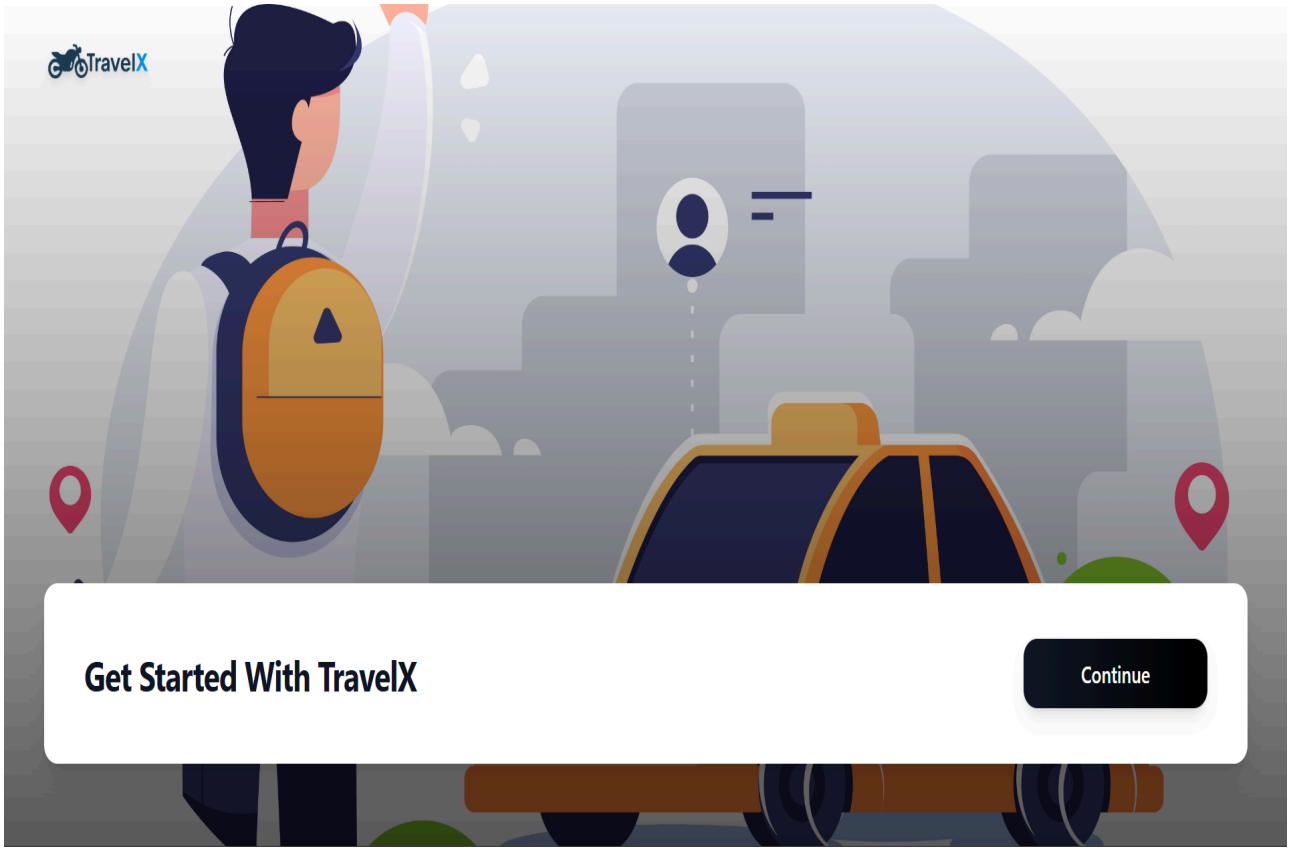
**The Settlement History Collection** records specific driver cash-out requests, storing withdrawal amounts and carefully masked bank account details for security.

**The Company Wallet Collection** acts as the singular, centralized operational account, constantly aggregating the total platform commissions and ride statistics to give admins an instantaneous overall revenue view.

MongoDB combined with Mongoose as the ODM will provide the ability to do CRUD operations on the back end and allow end users (both frontend and backend) to work together seamlessly. Through the use of a schema-less framework, MongoDB provides for future growth and changes in data formatting.

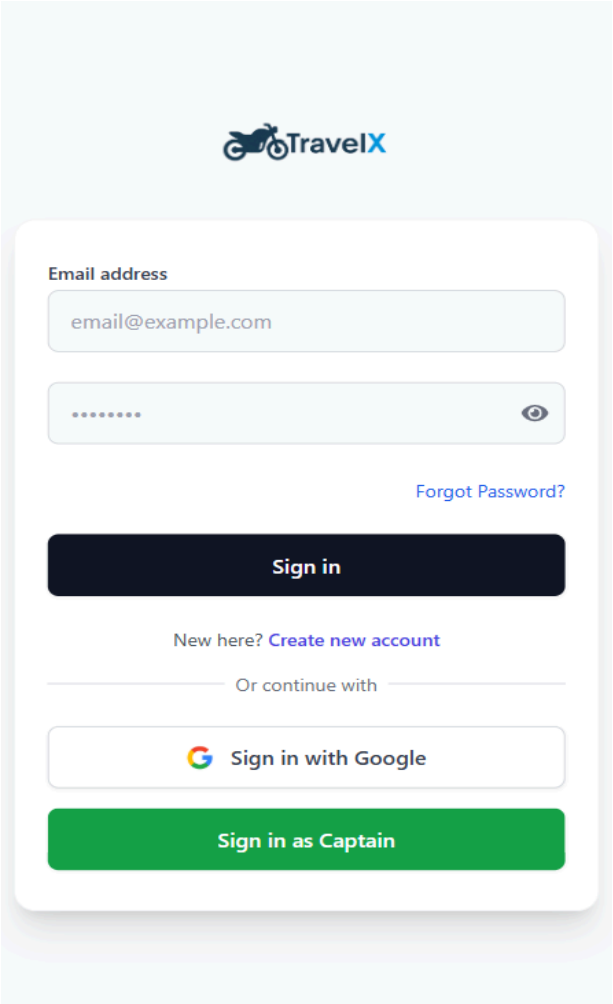
# 6. Program's Structure Analyzing and GUI Constructing

## 6.1 - Home Page

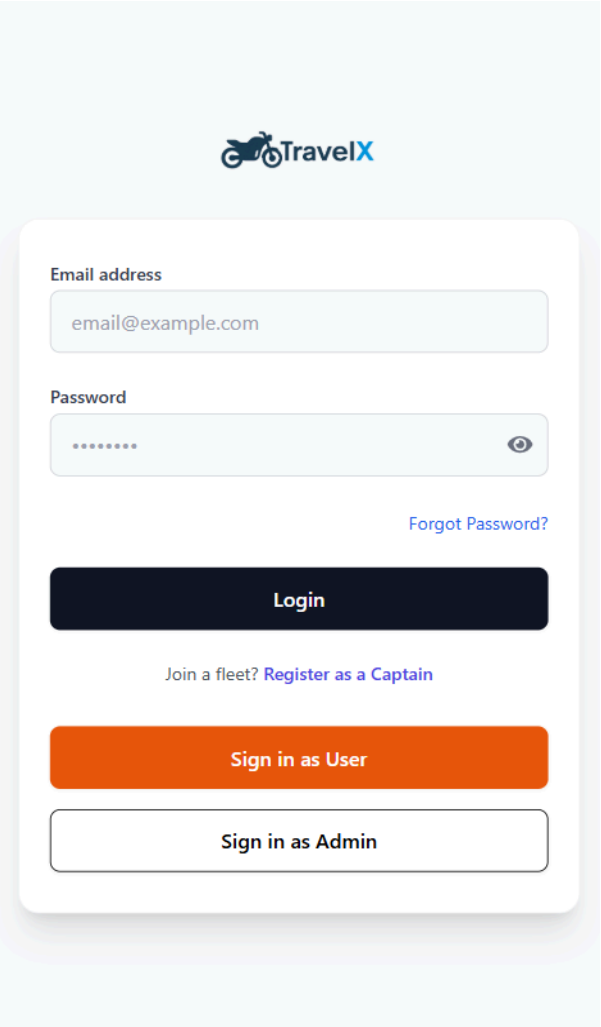


# Login Pages

## 6.2 - User Login Page

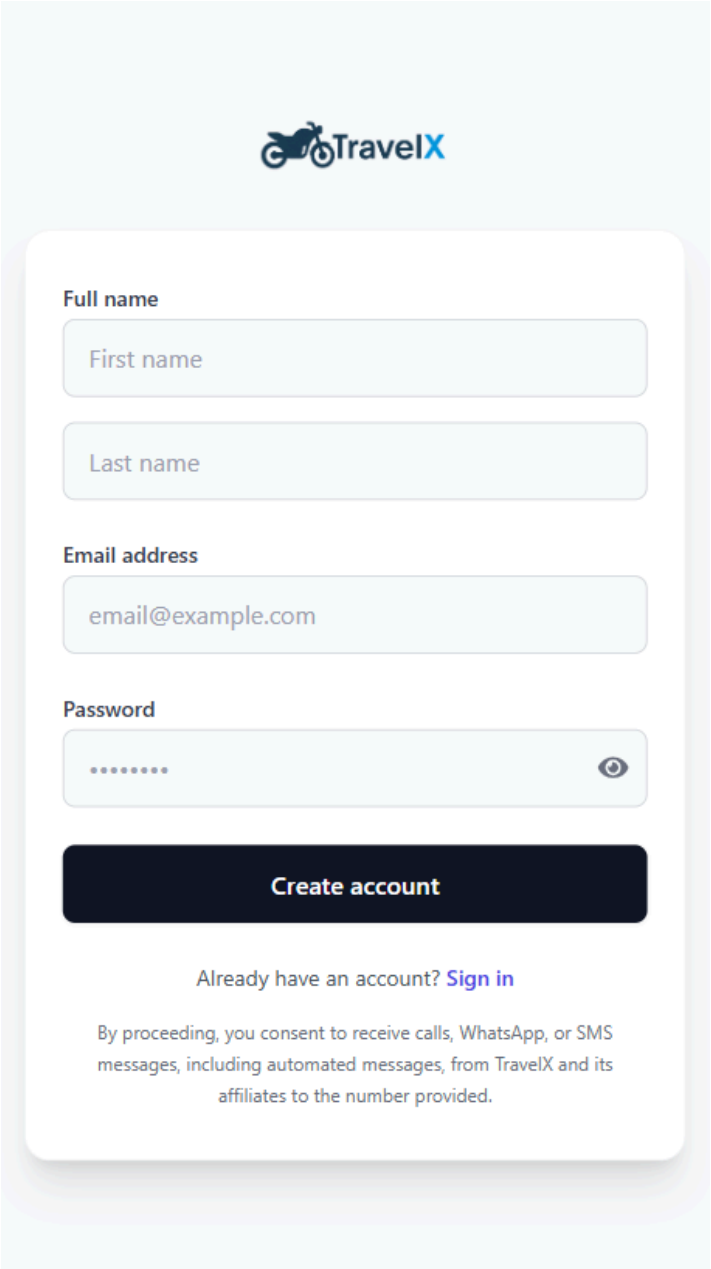



## 6.3 - Captain Login Page

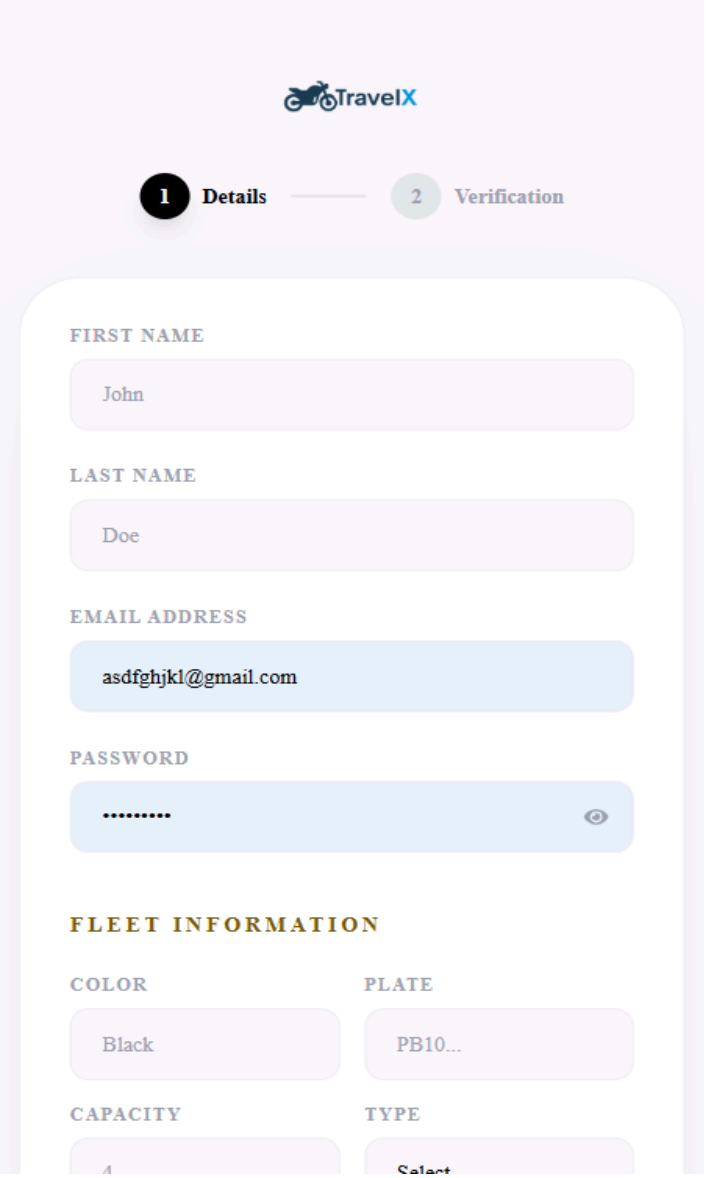


## SignUp Pages

### 6.4 - User SignUp Page



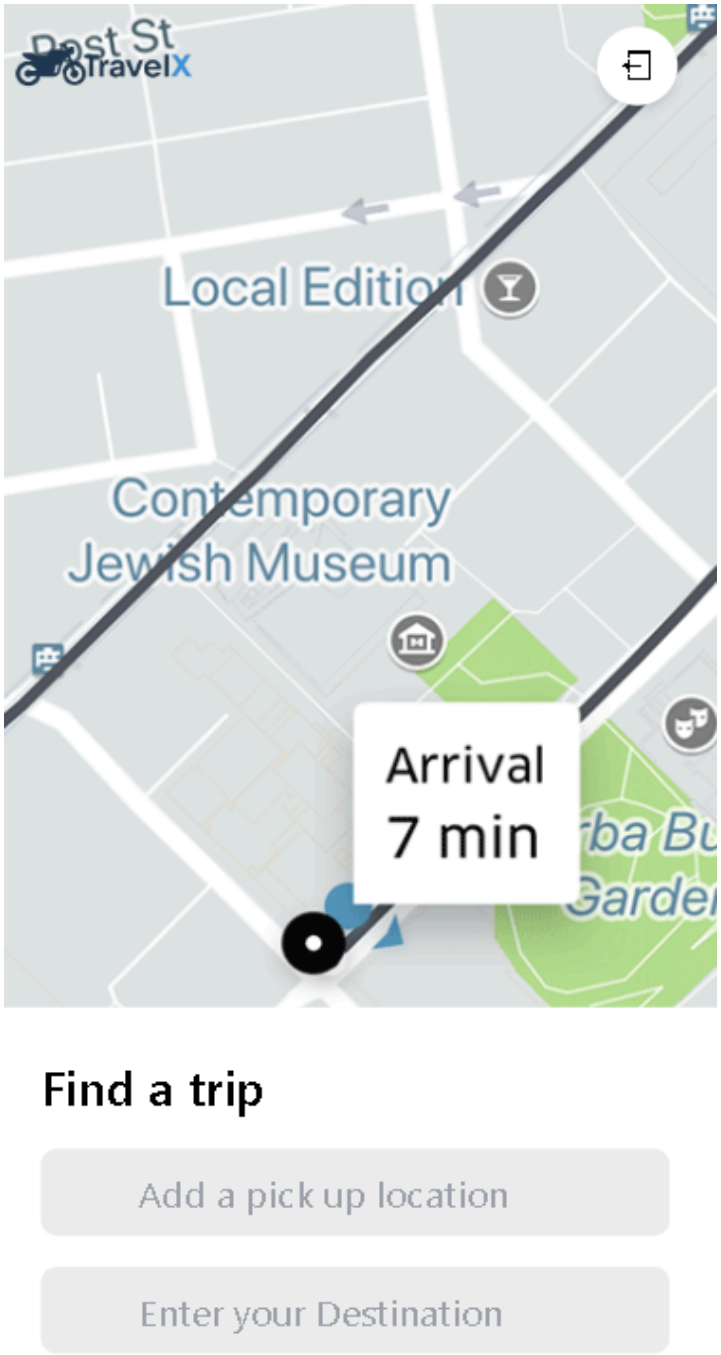

### 6.5 - Captain SignUp Page



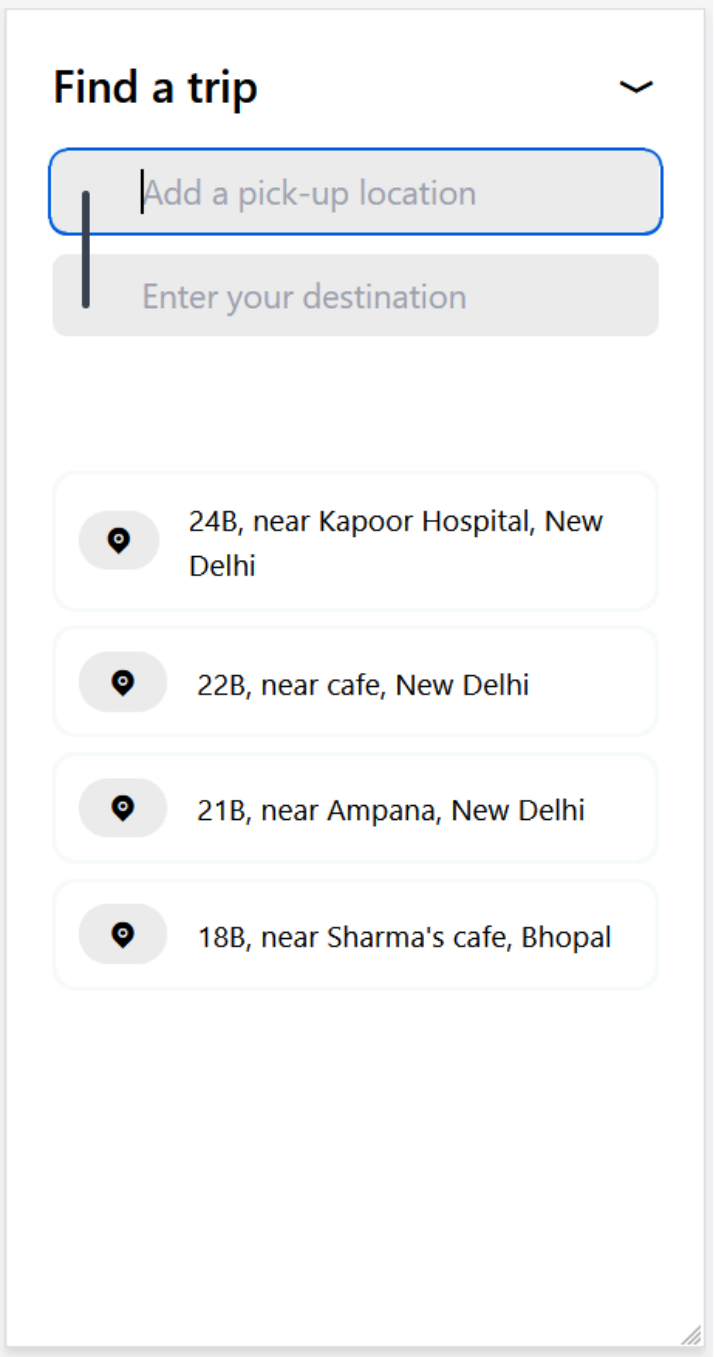

## User GUI

CHITKARA UNIVERSITY

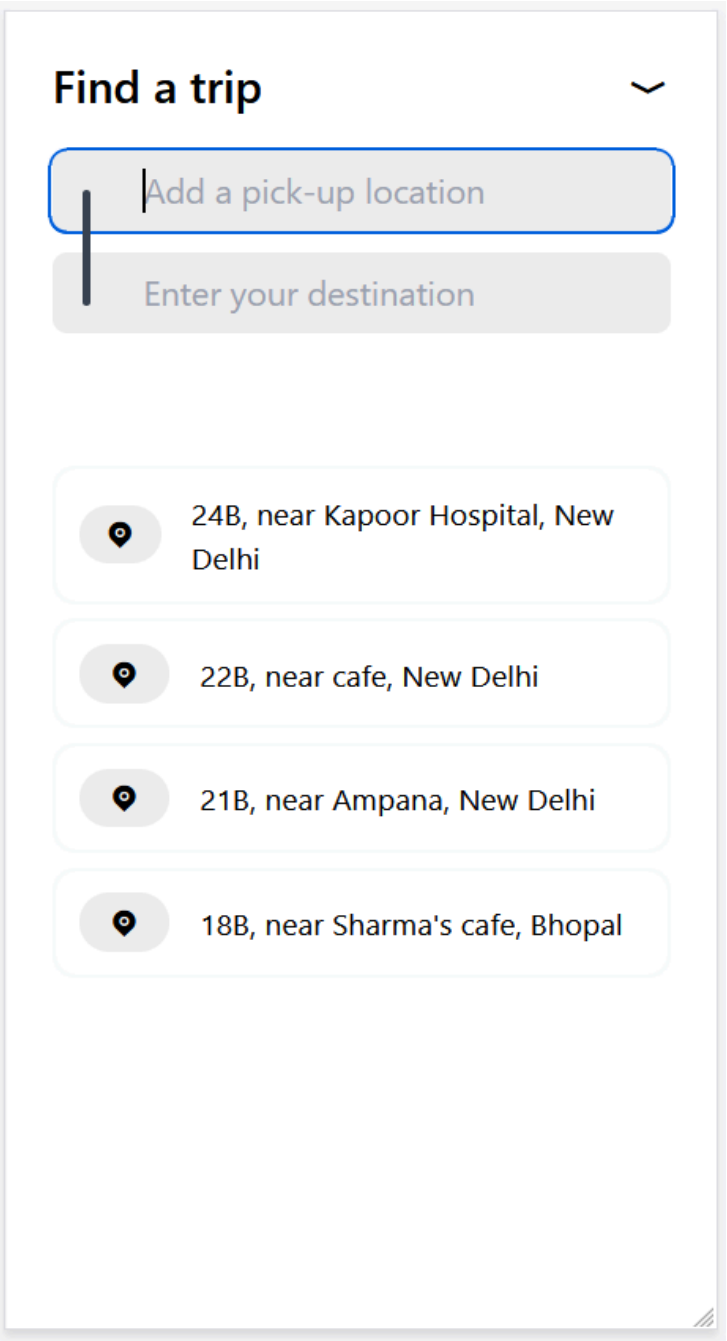
## 6.6 - User Home Page



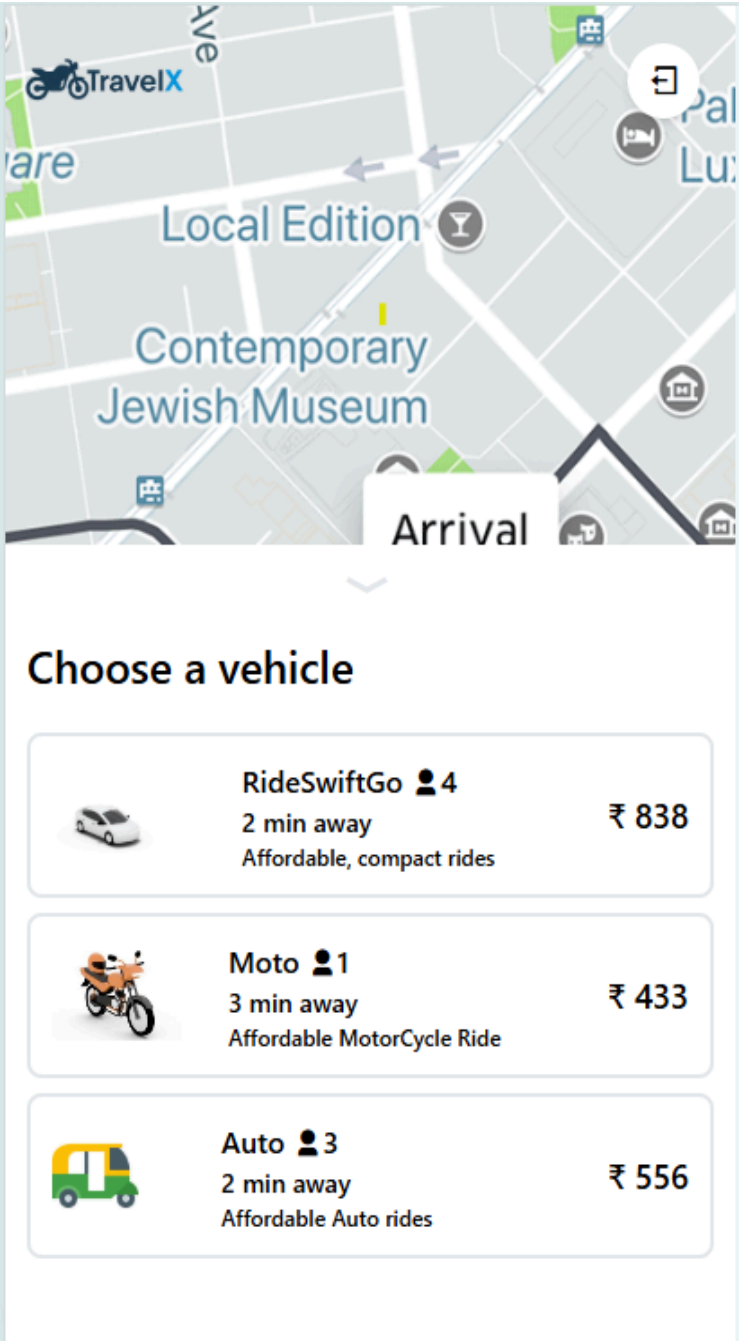

## 6.7 - Location Search Panel



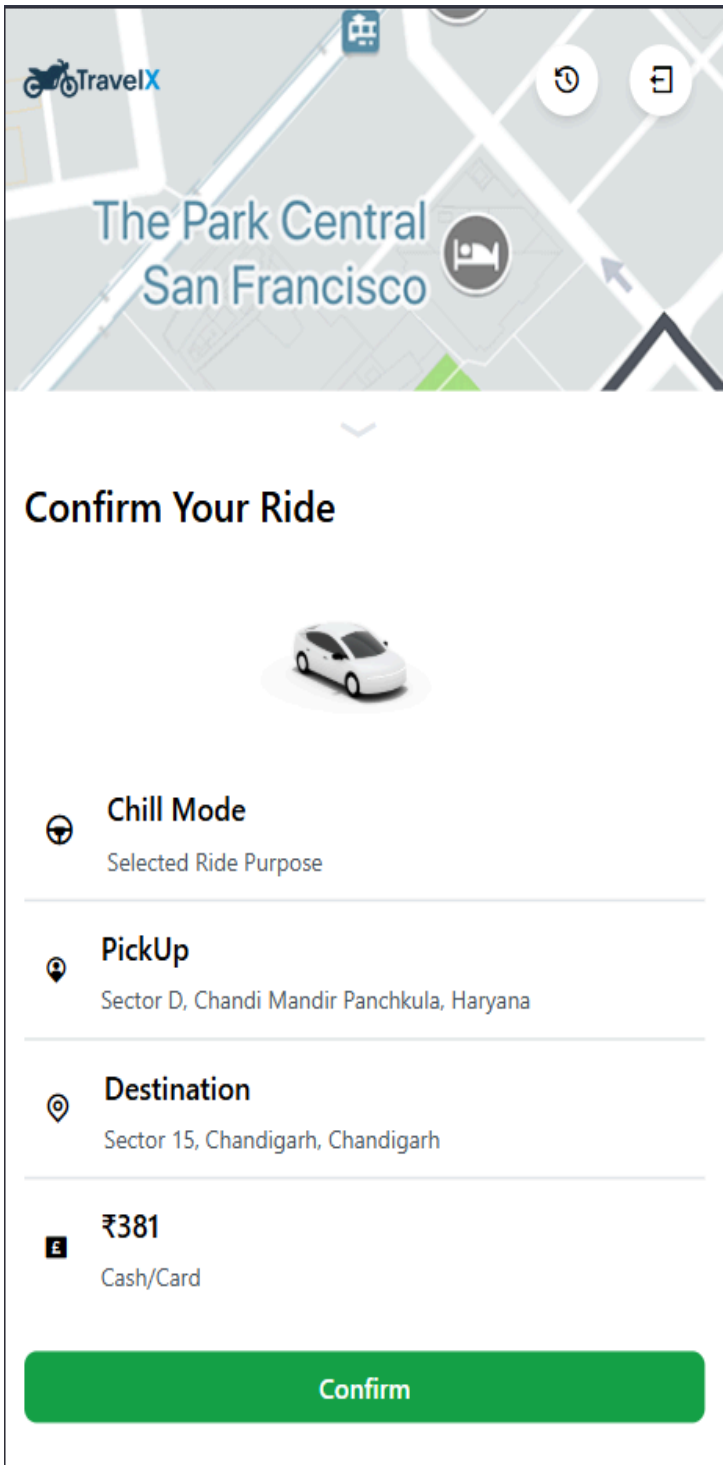
## 6.8 - User Location Panel



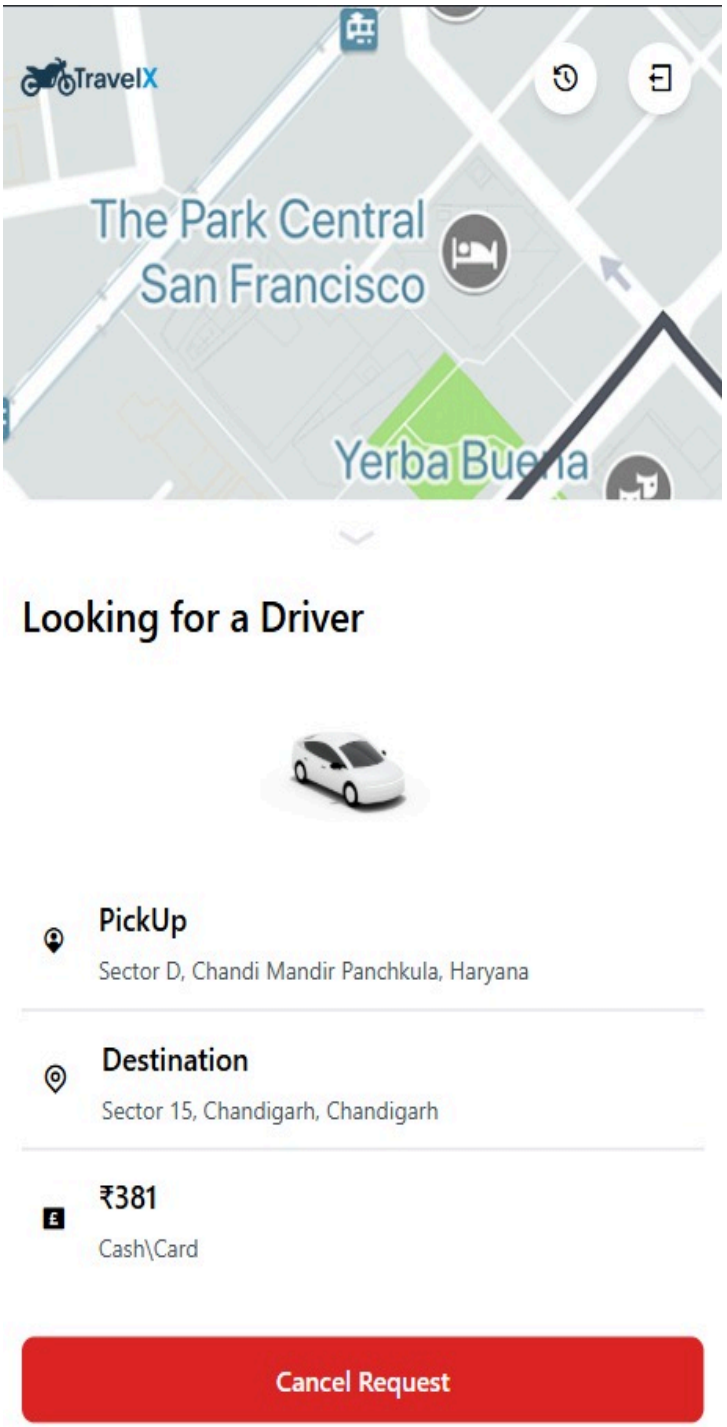

## 6.9 - User Vehicle Panel



## 6.10 - Confirmation Ride Panel

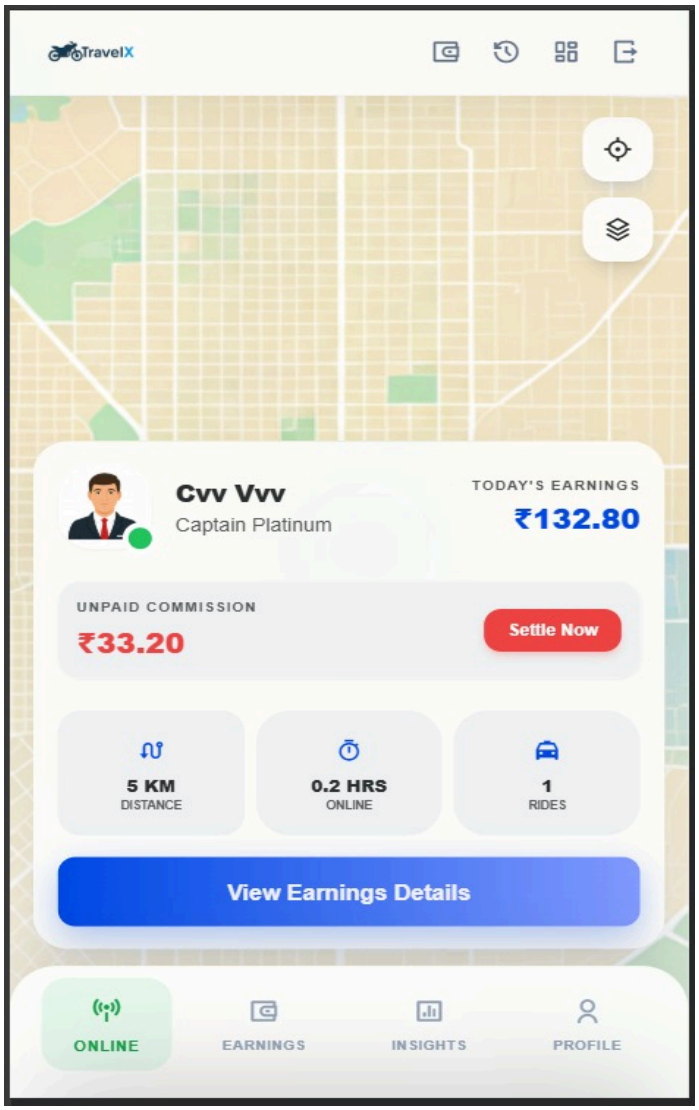



## 6.11 - Looking For Ride Panel

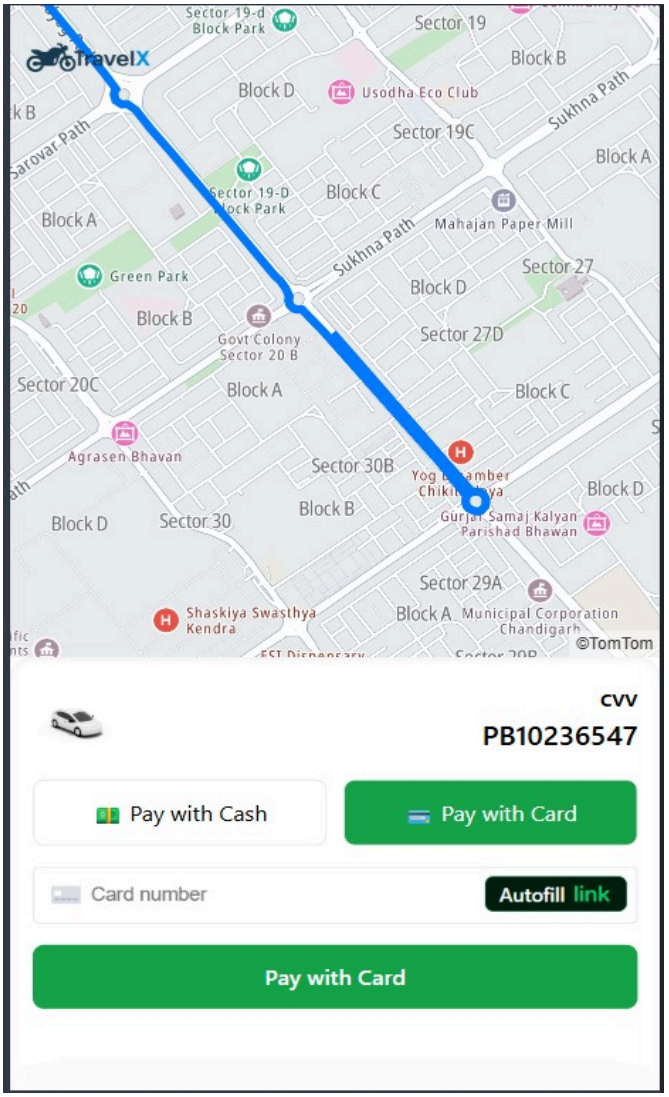


## Captain GUI

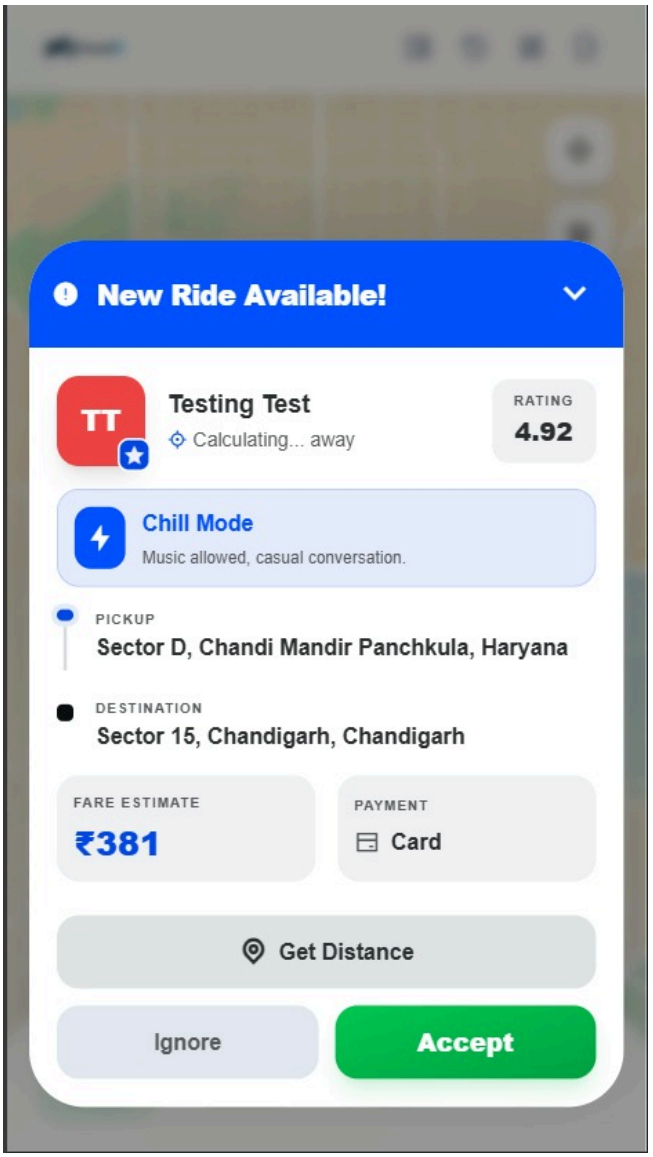
### 6.12 - Captain Home



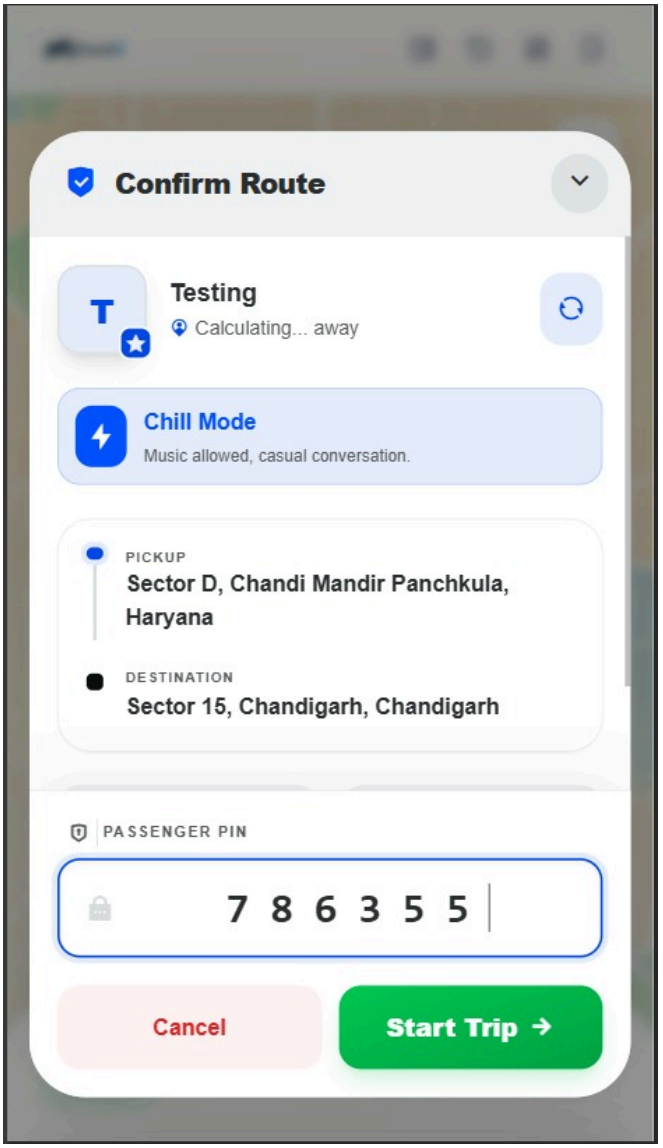

### 6.13 - Payment Page



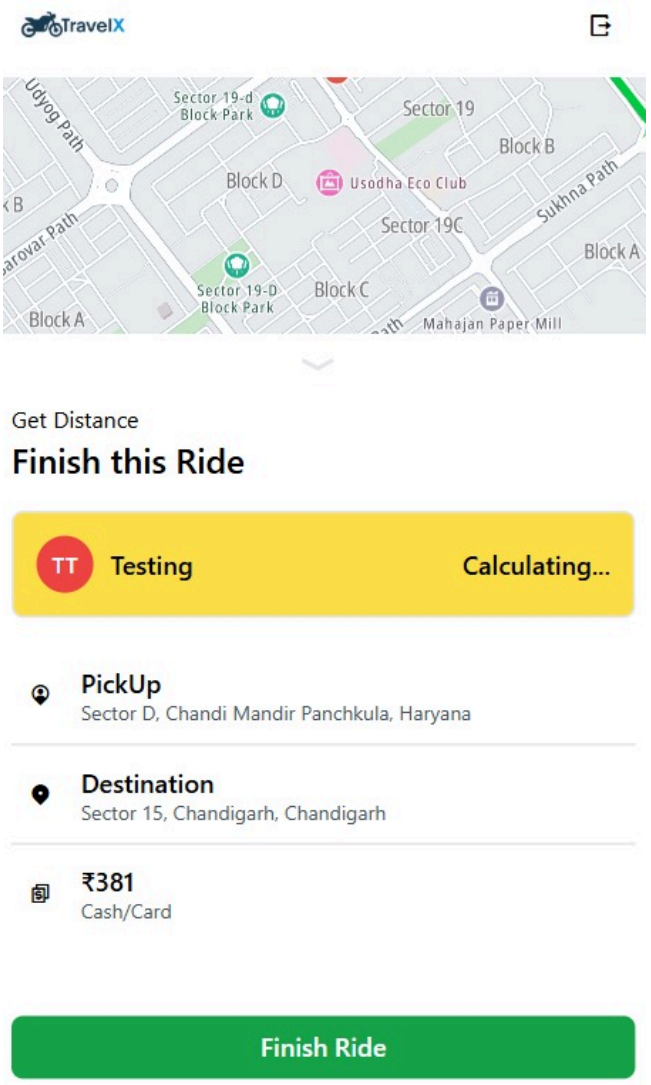
## 6.14 - New Ride Pop Up



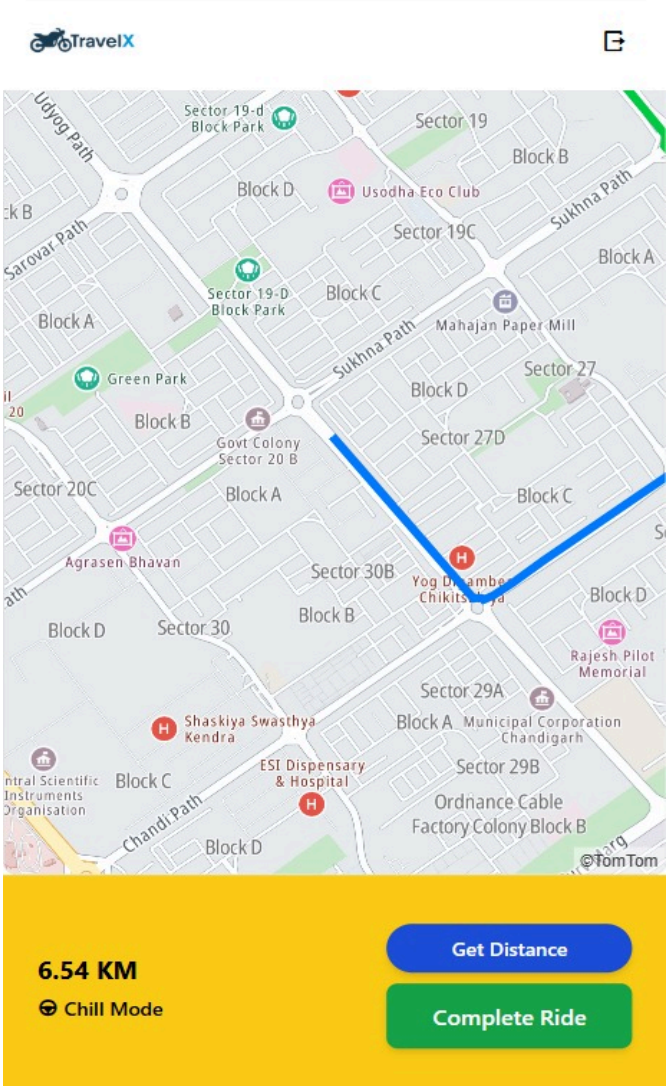

## 6.15 - Confirm Route



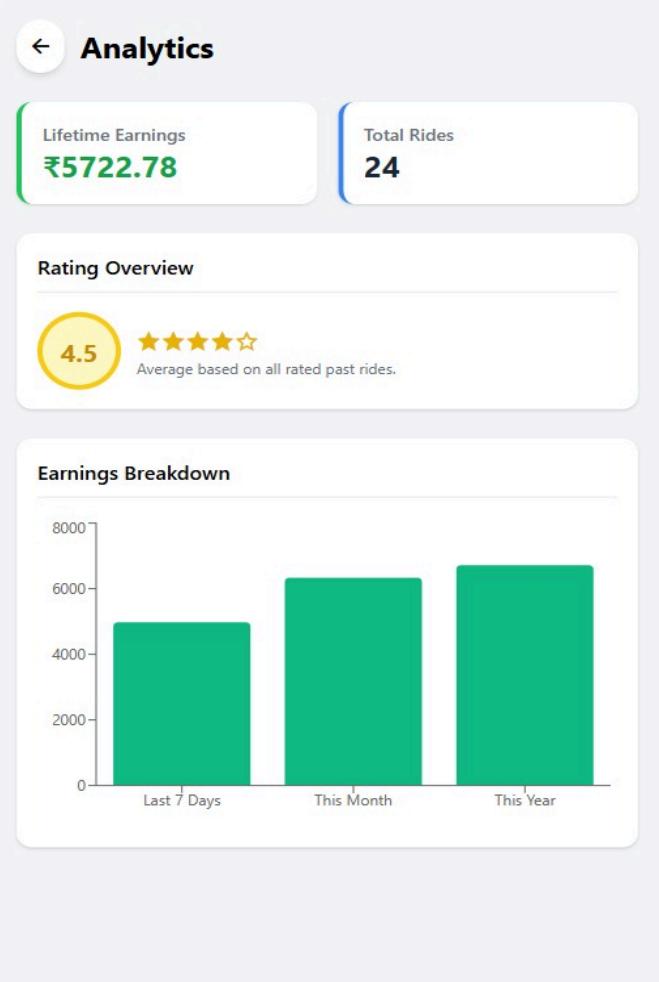
## 6.16 - Finish Ride Panel



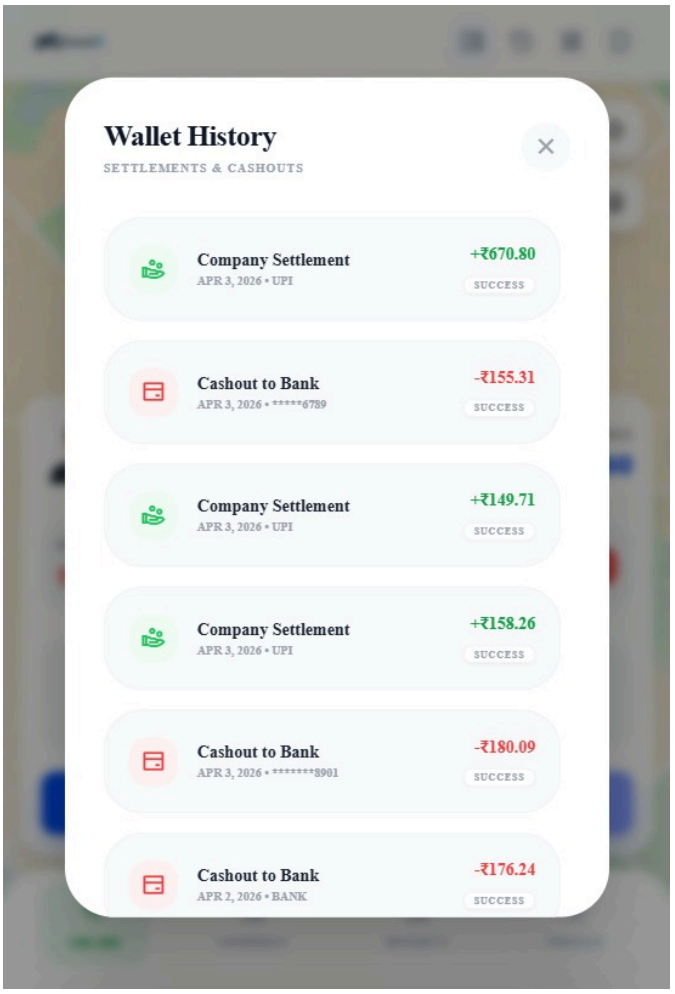

## 6.17 - Complete Ride Panel



## 6.18 - Captain Earnings

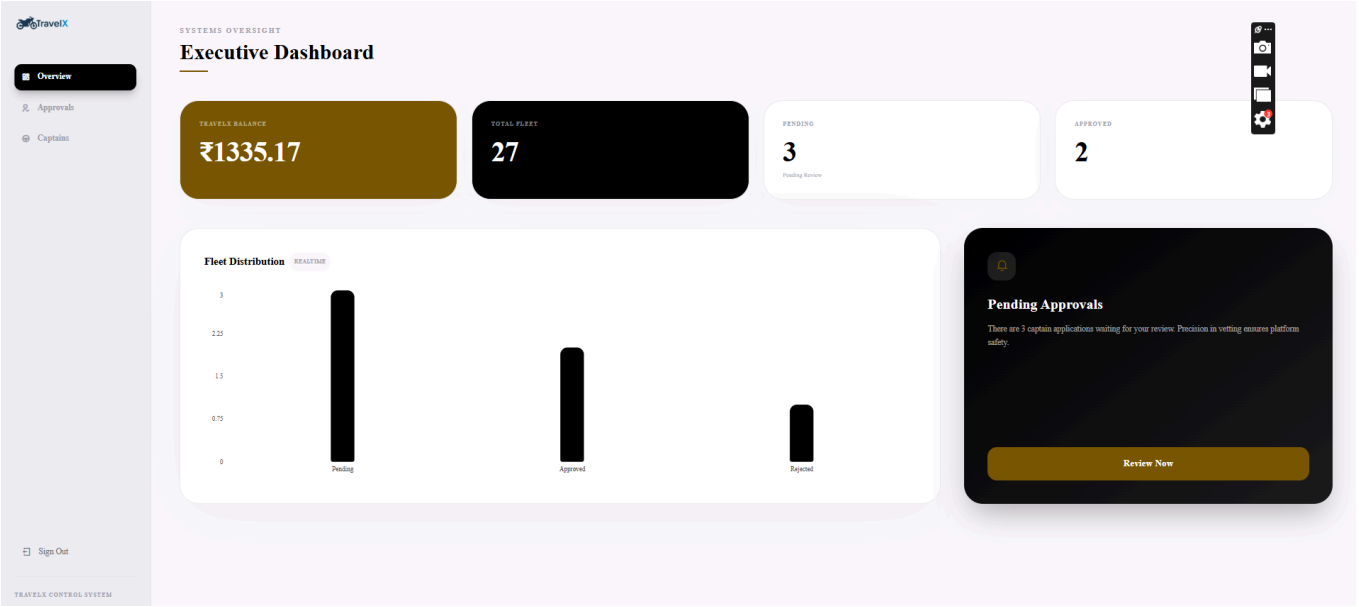



## 6.19 - Captain Wallet History

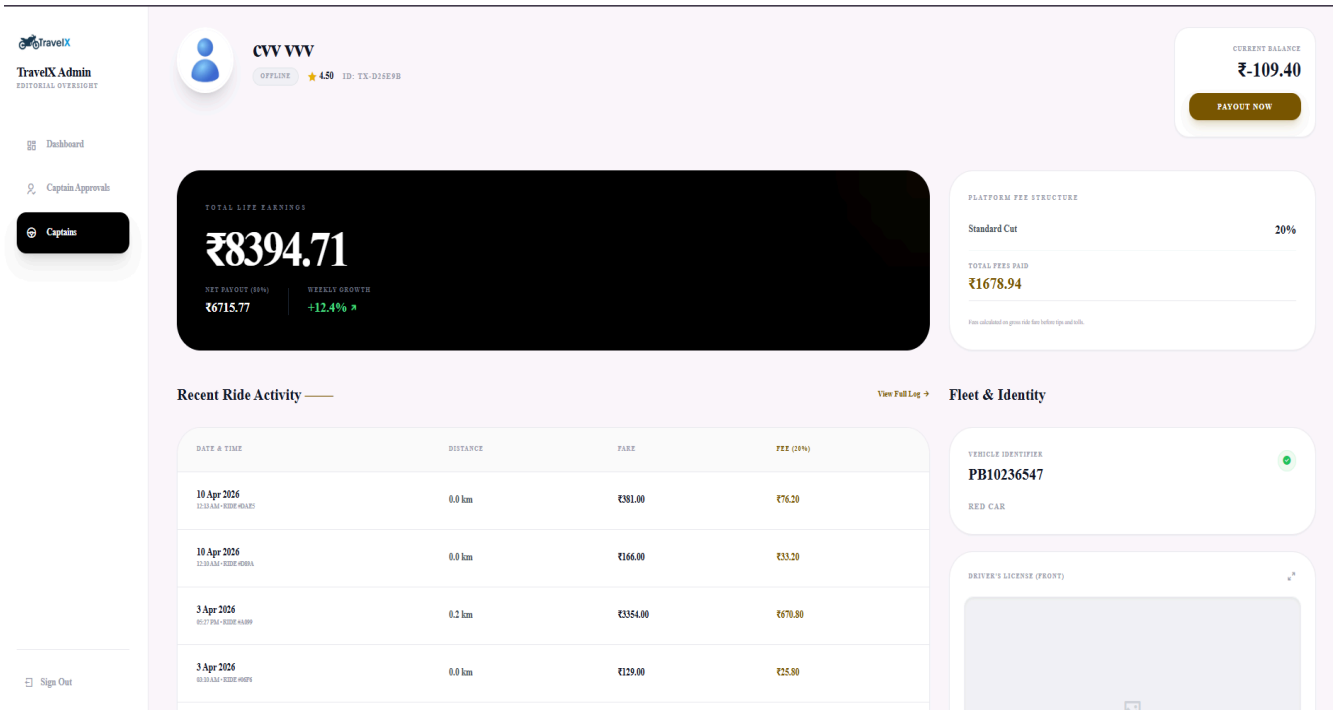


# Admin GUI

## 6.20 Admin Dashboard




## 6.21 Captain Control Panel



# 7. Limitations

- Dependency on the Internet: This application has a consistent dependency on internet connection all the time since this will help make sure that the rider as well as the driver can gain access to this platform all the time. When the geographical location of the driver or rider does not give them access to any kind of internet connection, then it may present itself as a significant challenge.

- Security Threats: Although the application makes use of different security measures like JWT authentication and data encryption, it doesn't mean that the application cannot be hacked by some form of cyber-attack such as a DDoS attack, data breach and many more.

- Scalability Concerns: Although the database used is efficient in processing a lot of data, it could face challenges when the amount of users rises due to inadequate optimization or lack of proper infrastructure.

- Offline Booking Not Possible: The user can't book rides using any other means such as through telephonic calls, making it impossible for people who may not be computer-literate to book rides using this application.

## 8.1 Conclusion

The TravelX app can be considered a good option for a scalable ride-hailing app. Users would be able to book a cab, follow the trip in real-time using maps, and make payments online conveniently. Captains would be able to manage their trips, set their availability for trips, manage their finances using wallet functionalities, settle their earnings, receive payouts and see their full transactions history.

One of the advantages of the system is its admin panel, which offers centralized control and enables administrators to manage riders and captains data, approve or deny driver registration, and analyze captain activities and earnings, captain profit generation, and the whole activity in the system. Additionally, the admins would be able to analyze the TravelX wallet activities to understand the flow of money.

Despite all the benefits that the TravelX system provides, there are still a number of problems such as reliance on an active internet connection, security concerns, and scalability problems. After making some improvements like adding AI algorithms to help with the allocation of rides, enhancing security measures and improving performance, the TravelX application can become a great alternative to public transport in our urban

## 8.2 Future Scope

- **Enhanced Security:** Implement advanced security measures such as multi-factor authentication, end-to-end encryption, and regular security audits to further safeguard against potential threats and ensure data integrity.
- **Scalability Improvements:** Optimize the system to handle larger datasets and higher traffic volumes efficiently. Implement load balancing and distributed database solutions to maintain performance and reliability during peak usage periods.
- **Advanced Analytics:** Integrate more sophisticated data analytics and reporting tools to provide deeper insights into ride demand, driver performance, and user engagement, helping stakeholders make informed decisions.
- **Blockchain Integration:** Investigate the potential of blockchain technology to add an extra layer of security and transparency to ride transactions, ensuring tamper-proof records and verifiable payment histories.
- **Mobile App Development:** Develop a mobile application version of the TravelX system to provide a more convenient and accessible ride-booking experience for users on smartphones and tablets.

# Bibliography/References


- **React Documentation** - Official React Documentation - Resource for understanding and implementing React features.
- **Express.js Documentation** - Official Express Documentation - Covers routing, middleware, and server configuration.
- **MongoDB Documentation** - Official MongoDB Documentation - Guide for setup, configuration, and database querying.
- **Node.js Documentation** - Official Node.js Documentation - Comprehensive documentation for server-side JavaScript runtime.
- **Tom Tom API** - Tom Tom API - Provides APIs for real-time location tracking and mapping services.
- **Socket.io Documentation** - Official Socket.io Documentation - Used for real-time, bidirectional event-based communication.
- **JWT Authentication** - JWT Documentation - Resource on JSON Web Tokens for authentication and authorization.